StockOpt – Agent Autonome de Gestion des Stocks pour l'Industrie 4.0

Nom de L'Equipe : Phoenix

Membres du projet :

- Hafsa ouattou
- Zineb bel achheb

Événement : ENSET Challenge – Phase Qualification

Catégorie : IA Agentique pour l'Industrie

DESCRIPTION DU PROJET	StockOpt est un agent IA autonome qui surveille les stocks d'un entrepôt, détecte les ruptures imminentes et passe automatiquement des commandes auprès des fournisseurs, sans intervention humaine.

1-Problématique

Dans les petites et moyennes industries, la gestion des stocks est encore souvent manuelle. Un opérateur humain doit vérifier régulièrement les niveaux de chaque référence et décider quand passer une commande.

Cette approche présente plusieurs limites :

- **Rupture de stock** → arrêt de la production, perte financière
- **Surstock** → argent immobilisé inutilement
- **Surveillance impossible 24h/24** → risque d'oubli ou de réaction tardive

Avec l'accélération des cadences industrielles, une solution automatisée et intelligente devient nécessaire.

2-Objectif du projet :

Concevoir un agent IA autonome capable de surveiller en continu les niveaux de stock, de détecter les seuils critiques, de décider de manière raisonnée de passer commande, et d'exécuter cette commande auprès d'un fournisseur, sans intervention humaine.

3-Exigences fonctionnelles:

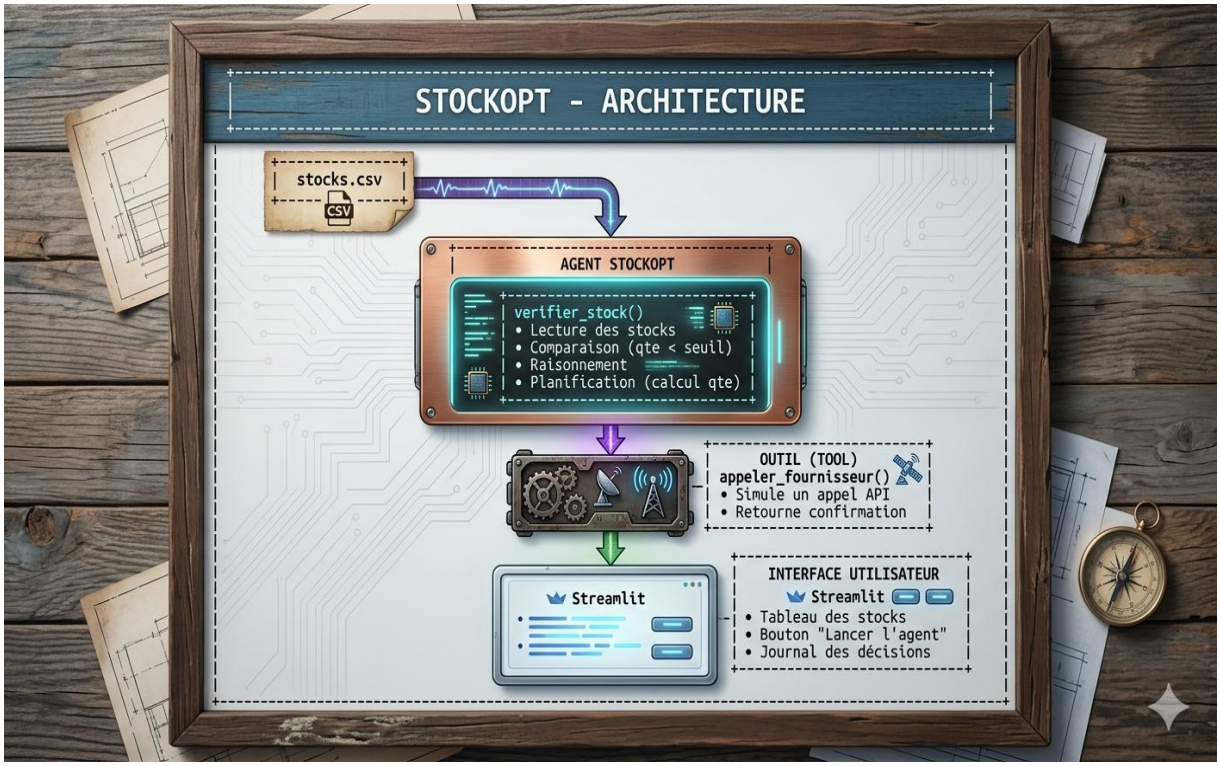
ID	Exigence fonctionnelle	Description
EF1	Lecture des stocks	L'agent lit les niveaux de stock depuis un fichier CSV (produit, quantité, seuil critique)
EF2	Détection de seuil critique	L'agent compare chaque stock avec son seuil et identifie les situations critiques
EF3	Raisonnement	L'agent calcule la quantité à commander (exemple : seuil × 3 ou stock minimum × 3)

EF4	Décision autonome	L'agent décide seul de passer commande ou non, sans validation humaine
EF5	Appel fournisseur	L'agent utilise un outil (tool) pour contacter une API fournisseur (simulée)
EF6	Journalisation	L'agent enregistre chaque décision (date, produit, quantité, statut) dans un log visible
EF7	Affichage temps réel	Un tableau de bord permet de visualiser stocks, seuils et historique des décisions

4-Exigences techniques:

ID	Exigence technique	Solution retenue
ET1	Architecture agentique	Agent unique avec raisonnement intégré
ET2	Interface utilisateur	Streamlit (tableau de bord simple et interactif)
ET4	Stockage des données	Fichier CSV (stock.csv) – pas de base de données complexe
ET5	API fournisseur	Fonction Python fictive (tools.py)
ET6	Langage de développement	Python 3.14.2
ET7	Versionnement	GitHub avec README clair

5-Architecture agentique :



1. Structure Globale de l'Architecture

L'architecture est composée de quatre blocs principaux interconnectés, orchestrant le flux d'informations :

1. **La Source de Données (stocks.csv)**
2. **L'Intelligence du Système (AGENT STOCKOPT)**
3. **L'Actionneur ou Outil (OUTIL (TOOL))**
4. **L'Interface Utilisateur (Streamlit)**

Le flux logique global est le suivant : l'Agent lit les données de stock, prend une décision, utilise un outil pour exécuter cette décision (comme passer une commande), puis rend compte de ses actions via l'interface utilisateur.

2. Analyse Détaillée des Composants

Voici une description détaillée de chaque module et de son rôle précis :

A. Source de Données (stocks.csv)

C'est le point de départ du système. Le fichier **stocks.csv** agit comme une base de données simplifiée. Il contient l'état actuel de l'inventaire, probablement sous forme de tableau avec des colonnes telles que le nom du produit, la quantité en stock, et éventuellement le seuil de réapprovisionnement.

B. AGENT STOCKOPT (Le Cœur Décisionnel)

C'est le composant le plus important, agissant comme le "cerveau" de l'application. Sa fonction principale est **verifier_stock()**. Il est responsable de l'analyse et de la planification. Il fonctionne en quatre étapes clés :

- **Lecture des stocks :** L'agent récupère les données brute du fichier stocks.csv.
- **Comparaison (qte < seuil) :** Pour chaque produit, il effectue un test logique. Il compare la quantité actuelle (qte) à un niveau de stock critique défini (seuil).
- **Raisonnement :** Sur la base de la comparaison, il décide si une action est nécessaire. Si qte < seuil, il conclut qu'un réapprovisionnement est indispensable.
- **Planification (calcul qte) :** Si une commande doit être passée, l'agent calcule précisément la quantité à commander pour ramener le stock à un niveau optimal (par exemple, pour atteindre un stock cible).

Une fois la décision prise et la quantité calculée, l'agent délègue l'exécution à l'outil.

C. OUTIL (TOOL) - appeler_fournisseur()

L'agent ne passe pas la commande directement. Il utilise un **"Outil"** dédié pour interagir avec le monde extérieur. Cet outil possède la fonction **appeler_fournisseur()**.

- **Rôle :** Son rôle est d'exécuter l'action technique de commande.

- **Simule un appel API :** Dans cette architecture, le "Tool" agit comme un wrapper autour d'une interface de programmation (API). Il envoie les détails de la commande (produit, quantité calculée par l'agent) au système du fournisseur.
- **Retourne confirmation :** Une fois l'appel API "réussi", l'outil retourne une confirmation à l'agent pour valider que l'action a été effectuée.

D. INTERFACE UTILISATEUR (Streamlit)

Ce bloc représente ce que l'utilisateur final voit et utilise. Il est construit avec **Streamlit**, un framework Python populaire pour créer des applications web de données. Il remplit trois fonctions essentielles :

- **Tableau des stocks :** Il affiche de manière lisible les données issues de stocks.csv, permettant à un humain de visualiser l'état de l'inventaire.
- **Bouton "Lancer l'agent" :** C'est le déclencheur manuel. L'utilisateur clique sur ce bouton pour démarrer le processus de vérification de l'agent.
- **Journal des décisions :** Après l'exécution de l'agent, cette section affiche un rapport d'activité. L'utilisateur peut y lire des messages comme : "Produit A : Stock OK", ou "Produit B : Commande de 50 unités passée au fournisseur (Confirmation ID: 123)".

- Tool use : appel fournisseu

6-Rôle des agents :

- **Agent Superviseur** : lit les stocks, compare aux seuils, identifie les alertes
- **Agent Actionneur** : reçoit l'alerte, décide la quantité, exécute la commande via l'outil

Comportement agentique démontré :

- Raisonnement : "Stock bas → risque rupture → je commande"
- Planification : "Je commande maintenant, je vérifierai plus tard"
- Utilisation d'outils : appel à l'API fournisseur

7-Fonctionnalités implémentées (MVP)

- Tableau de bord Streamlit avec 3 produits (vis, écrous, moteurs)
- Affichage des stocks et seuils critiques
- Exécution manuelle de l'agent (ou automatique toutes les X secondes)
- Détection automatique du seuil (stock < seuil)
- Calcul de la quantité à commander
- Appel à une API fournisseur fictive
- Journal des décisions (visible dans l'interface)
- Code source sur GitHub avec README

8-Démonstration (contenu de la vidéo) :

La vidéo de démonstration montre clairement :

1. Lancement de l'application Streamlit
2. Tableau de bord initial : vis = 18, seuil = 20
3. Clic sur "Exécuter l'agent"
4. Log affiché : "Vis = 18 < seuil 20 → alerte"
5. Log affiché : "Décision : commander 100 vis"
6. Log affiché : "Appel API fournisseur"
7. Log affiché : "Commande #123 acceptée"
8. Conclusion : aucune intervention humaine, l'agent a raisonné, décidé et agi seul

9-Difficultés rencontrées et solutions:

Difficulté	Solution apportée
L'agent oublie le contexte entre deux exécutions	Utilisation de la mémoire native de CrewAI
Temps de réponse du LLM trop long	Utilisation de Groq (inférence très rapide)
Interface trop complexe à coder	Choix de Streamlit (quelques lignes suffisent)
Simulation réaliste du fournisseur	Création d'une simple fonction Python avec retour JSON

10-Améliorations futures (post-hackathon):

- Prévision des ruptures basée sur la consommation passée
- Intégration à un vrai ERP (Odoo, SAP)
- Envoi de notifications par email ou SMS
- Support de plusieurs fournisseurs avec comparaison des prix
- Dashboard plus avancé (graphiques, historiques)

Conclusion:

StockOpt démontre qu'un agent IA autonome peut gérer les stocks industriels sans intervention humaine. L'agent raisonne, planifie, utilise des outils et prend des décisions seul. Le prototype est fonctionnel, la vidéo démo le prouve, et le code est accessible sur GitHub.